WEEK 2 · LESSON 7 OF 24 · HANDOUT

Decision Trees: Models You Can Read

Study time: **about 50 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=_L39rN6gz7Y — open the lesson online to watch inside the course.

🎓 **Go deeper:** StatQuest: Decision Trees, Part 2 — feature selection and missing data — https://www.youtube.com/watch?v=wpNl-JwwplA

🎓 **Go deeper:** SciPy 2018 tutorial: Machine Learning with scikit-learn, Part 2 (scikit-learn core developers) — https://www.youtube.com/watch?v=gK43gtGh49o

Your goal in this lesson: Explain how a decision tree chooses its questions, train and draw one in scikit-learn, and control its depth to stop overfitting.

A model made of questions

A **decision tree** works like a loan officer with a checklist: "Is the income above 150,000 RWF? Is the loan bigger than 190,000 RWF?" Each question splits the borrowers into two groups, until the groups are pure enough to give an answer. Anyone can read a small tree — that is why trees are loved in banks, hospitals and government.

Run the data and the split:

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(11)
n = 2000
income = rng.lognormal(12.2, 0.45, n).round(-3)
loan = rng.uniform(20_000, 300_000, n).round(-3)
months = rng.integers(1, 61, n)
late = rng.poisson(0.6, n)
salary = rng.random(n) < 0.55
z = -3.2 + 2.5 * (loan / income) - 0.03 * months + 0.8 * late - 0.9 * salary
default = (rng.random(n) < 1 / (1 + np.exp(-z))).astype(int)
loans = pd.DataFrame({"income_rwf": income, "loan_rwf": loan, "months_customer": months,
                      "late_payments": late, "salaried": salary.astype(int),
                      "defaulted": default})
print(loans.shape, "default rate:", round(loans["defaulted"].mean(), 3))
loans.head()
```

```
from sklearn.model_selection import train_test_split

X = loans.drop(columns="defaulted")
y = loans["defaulted"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
```

Train and read a small tree

```
from sklearn.tree import DecisionTreeClassifier, export_text

tree = DecisionTreeClassifier(max_depth=2, random_state=42)
tree.fit(X_train, y_train)
print(export_text(tree, feature_names=list(X.columns), show_weights=True))
print("Test accuracy:", tree.score(X_test, y_test))
```

Read the output from the top:

1. The first question is **income ≤ 148,500 RWF?** — the most useful single question.
2. For lower-income borrowers, the next question is **loan ≤ 191,500 RWF?** Of those with bigger loans, 122 of 154 defaulted, so that leaf predicts **1 (default)**.
3. For higher-income borrowers, the loan size is checked again; their leaves predict **0**.

`weights: [32, 122]` means 32 repaid and 122 defaulted in that leaf. With only four leaves, this tree already scores about **0.83** on the test set.

Draw it

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

plt.figure(figsize=(12, 5))
plot_tree(tree, feature_names=list(X.columns), class_names=["repaid", "default"],
          filled=True, rounded=True)
plt.show()
```

How the tree chooses a question: Gini impurity

At each step the tree tries **every feature and every threshold** and picks the split that makes the groups **purest**. Purity is measured with **Gini impurity**:

> *Gini = 1 − (share of class 0)² − (share of class 1)²*

- A group with only one class has Gini **0** (perfectly pure).
- A 50/50 group has Gini **0.5** (as mixed as possible).
- Our whole training set (21% defaulters) has Gini = $1 - 0.79^2 - 0.21^2 \approx$ **0.33**.

```
def gini(n0, n1):
    p0, p1 = n0 / (n0 + n1), n1 / (n0 + n1)
    return round(1 - p0**2 - p1**2, 3)

print(gini(1183, 317))   # all training borrowers
print(gini(468, 22))     # high income, small loan: very pure
print(gini(32, 122))     # low income, big loan: mostly defaulters
```

The tree keeps the split that lowers the **weighted average** Gini the most. **Entropy** (`criterion="entropy"`) is another purity measure; results are usually similar.

The danger: trees love to memorise

```
for depth in [2, 4, 6, 10, None]:
    t = DecisionTreeClassifier(max_depth=depth, random_state=42).fit(X_train, y_train)
    print(depth, "train", round(t.score(X_train, y_train), 3),
          "test", round(t.score(X_test, y_test), 3), "leaves", t.get_n_leaves())
```

With no limit (`None`) the tree grows until every leaf is pure: **100%** training accuracy, about **0.76** on the test set, and more than 200 leaves. It has memorised the noise — classic **overfitting**. Shallow trees generalise much better here.

Ways to keep a tree in check (called **pruning**):

- `max_depth` — the maximum number of questions in a row.
- `min_samples_leaf` — each leaf must contain at least this many examples (for example 20).
- `max_leaf_nodes` — a limit on the number of leaves.

Feature importance

```
print(pd.Series(tree.feature_importances_, index=X.columns).round(2))
```

Importance shows how much each feature reduced impurity. The small tree uses only income and loan size, so every other feature gets 0. Careful: importance tells you what the **model** used, not what **causes** default.

Trees for numbers

`DecisionTreeRegressor` predicts a number: each leaf gives the **average** value of its training examples. The rent of a home could be predicted by questions such as "size > 100 m²? furnished?".

Strengths and weaknesses

Strengths	Weaknesses
Easy to read and explain	Overfit easily when deep
No scaling needed	Unstable: small data changes can change the whole tree
Handle numbers and categories, and curved patterns	A single tree is rarely the most accurate model

The weaknesses are exactly what **random forests** fix in the next lesson.

Key terms

- **Decision tree** — a model made of yes/no questions.
- **Node / leaf** — a question / a final answer.
- **Gini impurity** — how mixed a group is (0 = pure).
- **Pruning** — limiting a tree's growth to avoid overfitting.
- **Feature importance** — how much each feature helped the splits.

Remember this: A decision tree asks the questions that make groups purest; it is easy to explain but must be kept shallow to avoid memorising noise.

Name: Date:

Activity

Task 1

Train a tree with `max_depth=3`. Write its rules as three plain-English sentences a loan officer could follow.

Task 2

Try `min_samples_leaf=20` with no depth limit. Compare train and test accuracy with the unlimited tree.

Task 3

Compute by hand the Gini of a leaf with 163 repaid and 48 defaulted. Check with the `gini` function.

Task 4

Train a `DecisionTreeRegressor(max_depth=3)` on the homes data. Which feature does it ask about first?

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What does each internal node of a decision tree do?
   - A. Removes rows
   - B. Predicts a number
   - C. Scales the data
   - D. Asks a yes/no question about a feature
2. What is the Gini impurity of a group that contains only one class?
   - A. 0.5
   - B. 0
   - C. It depends on the size
   - D. 1
3. A tree with no depth limit gets 100% on training data and 76% on test data. What is happening?
   - A. Data leakage
   - B. Perfect learning
   - C. Underfitting
   - D. Overfitting: it memorised the training data
4. Which setting does NOT help limit a tree's growth?
   - A. max_leaf_nodes
   - B. max_depth
   - C. min_samples_leaf
   - D. random_state
5. Do decision trees need scaled features?
   - A. Only with Gini
   - B. No, they only compare values against thresholds
   - C. Yes, always
   - D. Only for regression
6. What does high feature importance tell you?
   - A. The model relied on that feature for its splits
   - B. The feature should be deleted
   - C. The feature has missing values
   - D. The feature causes the outcome

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.